Assignment

Bachelor of Computer Applications

Semester III

Database Management System

Topic: Advanced SQL Queries Using Joins and Subqueries

BY

Name: M Blessy

Registration Number: 2511021240067

DEPARTMENT OF COMPUTER APPLICATIONS

Alliance university

Chikkahagade cross, Anekal, Bangaluru 562106

Advanced SQL Queries Using Joins and Subqueries

1. Objective

To retrieve meaningful information from the Hospital Management System using SQL joins and subqueries. The queries combine related records from multiple tables and generate useful reports about appointments, admissions, prescriptions, bills, laboratory tests, and doctors.

2. Introduction

Joins are used to retrieve related information from two or more tables. Subqueries are queries written inside another SQL query. They help retrieve information based on conditions or calculated values.

In the Hospital Management System, joins and subqueries help connect patient records with doctors, appointments, admissions, prescriptions, medicines, laboratory tests, and bills.

3. Reports Using Joins

Query 1: Appointment Report

SQL command:

```
SELECT
   a.appointment_id,
   p.name AS patient_name,
   d.name AS doctor_name,
   d.specialization,
   a.appointment_date,
   a.status
FROM Appointment a
INNER JOIN Patient p
   ON a.patient_id = p.patient_id
INNER JOIN Doctor d
   ON a.doctor_id = d.doctor_id
WHERE a.appointment_id BETWEEN 7001 AND 7004;
```

Explanation: This query joins the Appointment, Patient, and Doctor tables to display appointment IDs, patient names, doctor names, specializations, dates, and appointment statuses. It displays appointments whose IDs are between 7001 and 7004.

appointment_id	patient_name	doctor_name	specialization	appointment_date	status
7001	Aarav Mehta	Priya Sharma	Cardiology	2026-10-26	Scheduled
7004	Meera Joseph	Priya Sharma	Cardiology	2026-10-28	Scheduled
7002	Diya Nair	Rahul Kumar	Pediatrics	2026-10-26	Scheduled
7003	Kiran Kumar	Ananya Rao	General Medicine	2026-10-27	Completed

Query 2: Patient Admission Report

SQL command:

```
SELECT
   a.admission_id,
   p.name AS patient_name,
   r.room_number,
   r.room_type,
   d.name AS doctor_name,
   a.admission_date,
   a.discharge_date,
   a.status
FROM Admission a
INNER JOIN Patient p
   ON a.patient_id = p.patient_id
INNER JOIN Room r
   ON a.room_id = r.room_id
INNER JOIN Doctor d
   ON a.doctor_id = d.doctor_id
WHERE a.admission_id BETWEEN 7001 AND 7002;
```

Explanation: This query joins the Admission, Patient, Room, and Doctor tables. It displays the patient's name, assigned room, doctor, admission date, discharge date, and admission status.

admission_id	patient_name	room_number	room_type	doctor_name	admission_date	discharge_date	status
7001	Aarav Mehta	C-201	Private	Priya Sharma	2026-10-20	NULL	Admitted
7002	Kiran Kumar	G-102	General	Ananya Rao	2026-10-18	2026-10-20	Discharged

Query 3: Prescription and Medicine Report

SQL command:

```
SELECT
   pr.prescription_id,
   p.name AS patient_name,
   d.name AS doctor_name,
   m.medicine_name,
   pd.dosage,
   pd.duration_days,
   pd.quantity
FROM Prescription pr
INNER JOIN Patient p
   ON pr.patient_id = p.patient_id
INNER JOIN Doctor d
   ON pr.doctor_id = d.doctor_id
INNER JOIN Prescription_Detail pd
   ON pr.prescription_id = pd.prescription_id
INNER JOIN Medicine m
   ON pd.medicine_id = m.medicine_id
WHERE pr.prescription_id BETWEEN 7001 AND 7002;
```

Explanation: This query combines five tables to show the medicines associated with prescriptions, including patient and doctor names, dosage, duration, and quantity.

prescription_id	patient_name	doctor_name	medicine_name	dosage	duration_days	quantity
7001	Aarav Mehta	Priya Sharma	Vitamin C Tablets	One tablet daily	10	10
7002	Diya Nair	Rahul Kumar	Paracetamol 500 mg	As directed	3	6
7002	Diya Nair	Rahul Kumar	Antacid Tablets	As directed	5	10

Query 4: Patient Billing Report

SQL command:

```
SELECT
   b.bill_id,
   p.name AS patient_name,
   a.appointment_id,
   ad.admission_id,
   b.bill_date,
   b.total_amount,
   b.payment_status,
   b.payment_mode
FROM Bill b
INNER JOIN Patient p
   ON b.patient_id = p.patient_id
LEFT JOIN Appointment a
   ON b.appointment_id = a.appointment_id
LEFT JOIN Admission ad
   ON b.admission_id = ad.admission_id
WHERE b.bill_id BETWEEN 7001 AND 7003;
```

Explanation: This query joins the Bill table with Patient, Appointment, and Admission. The LEFT JOIN operations allow the report to retain a bill even when its appointment or admission reference is NULL.

bill_id	patient_name	appointment_id	admission_id	bill_date	total_amount	payment_status	payment_mode
7001	Aarav Mehta	7001	7001	2026-10-26	3800.00	Pending	UPI
7002	Diya Nair	7002	NULL	2026-10-26	650.00	Paid	Cash
7003	Kiran Kumar	7003	7002	2026-10-20	2400.00	Paid	Card

Query 5: Laboratory Test Report

SQL command:

```
SELECT
   l.test_id,
   p.name AS patient_name,
   d.name AS doctor_name,
   l.test_name,
   l.test_date,
   l.result,
   l.status
FROM Lab_Test l
INNER JOIN Patient p
   ON l.patient_id = p.patient_id
INNER JOIN Doctor d
   ON l.doctor_id = d.doctor_id
WHERE l.test_id BETWEEN 7001 AND 7003;
```

Explanation: This query joins the Lab_Test, Patient, and Doctor tables to display laboratory test details along with the patient and doctor names, test date, result, and status.

test_id	patient_name	doctor_name	test_name	test_date	result	status
7001	Aarav Mehta	Priya Sharma	ECG	2026-10-26	Awaiting report	Pending
7002	Diya Nair	Rahul Kumar	CBC	2026-10-26	Within expected range	Completed
7003	Kiran Kumar	Ananya Rao	Blood Glucose	2026-10-27	Awaiting report	Pending

Query 6: Appointment Count for Each Doctor

SQL command:

```
SELECT
   d.doctor_id,
   d.name AS doctor_name,
   COUNT(a.appointment_id) AS total_appointments
FROM Doctor d
LEFT JOIN Appointment a
   ON d.doctor_id = a.doctor_id
GROUP BY d.doctor_id, d.name;
```

Explanation: This query uses a LEFT JOIN to include all doctors, counts their appointments, and groups the results by doctor. A doctor without appointments will also appear with a count of zero.

doctor_id	doctor_name	total_appointments
1	Dr. Arun	1
7001	Priya Sharma	2
7002	Rahul Kumar	1
7003	Ananya Rao	1

4. Reports Using Subqueries

Query 7: Doctors Charging Above the Average Fee

SQL command:

```
SELECT doctor_id, name, consultation_fee
FROM Doctor
WHERE consultation_fee > (
   SELECT AVG(consultation_fee)
   FROM Doctor
);
```

Explanation: The subquery calculates the average consultation fee of all doctors. The outer query displays only doctors whose consultation fees are greater than that average.

doctor_id	name	consultation_fee
7001	Priya Sharma	800.00

Query 8: Patients Having Scheduled Appointments

SQL command:

```
SELECT patient_id, name
FROM Patient p
WHERE EXISTS (
   SELECT 1
   FROM Appointment a
   WHERE a.patient_id = p.patient_id
     AND a.status = 'Scheduled'
     AND a.appointment_id BETWEEN 7001 AND 7004
);
```

Explanation: The EXISTS subquery checks whether each patient has a scheduled appointment with an ID between 7001 and 7004. The outer query displays the matching patient IDs and names.

patient_id	name
7001	Aarav Mehta
7002	Diya Nair
7004	Meera Joseph

Query 9: Departments With More Doctors Than the Average Department Count

SQL command:

```
SELECT
   d.department_id,
   d.department_name,
   COUNT(doc.doctor_id) AS total_doctors
FROM Department d
INNER JOIN Doctor doc
   ON d.department_id = doc.department_id
GROUP BY d.department_id, d.department_name
HAVING COUNT(doc.doctor_id) > (
   SELECT AVG(doctor_count)
   FROM (
      SELECT COUNT(*) AS doctor_count
      FROM Doctor
      GROUP BY department_id
   ) AS department_counts
);
```

Explanation: The inner subquery counts the doctors in each department and calculates the average department count. The outer query displays departments whose doctor count is greater than that average.

department_id	department_name	total_doctors
1	General Medicine	2

Query 10: Patients Whose Total Bills Exceed the Average Bill Amount

SQL command:

```
SELECT
   p.patient_id,
   p.name,
   SUM(b.total_amount) AS total_bills
FROM Patient p
INNER JOIN Bill b
   ON p.patient_id = b.patient_id
GROUP BY p.patient_id, p.name
HAVING SUM(b.total_amount) > (
   SELECT AVG(total_amount)
   FROM Bill
);
```

Explanation: The subquery calculates the average amount of an individual bill. The outer query calculates the total bill amount for each patient and displays patients whose total exceeds that average.

patient_id	name	total_bills
7001	Aarav Mehta	3800.00
7003	Kiran Kumar	2400.00

5. Conclusion

The advanced SQL queries demonstrate how joins and subqueries retrieve meaningful information from the Hospital Management System. The reports combine related data from multiple tables and provide information about appointments, admissions, prescriptions, billing, laboratory tests, and doctor workloads. These queries help organize hospital information and support reporting and decision-making.